

Activity_ Course 7 Salifort Motors project lab

June 8, 2026

1 Capstone project: Providing data-driven suggestions for HR

[]:

1.1 Description and deliverables

This capstone project is an opportunity for you to analyze a dataset and build predictive models that can provide insights to the Human Resources (HR) department of a large consulting firm.

Upon completion, you will have two artifacts that you will be able to present to future employers. One is a brief one-page summary of this project that you will present to external stakeholders as the data professional in Salifort Motors. The other is a complete code notebook provided here. Based on your prior coursework, select one method to complete this project: use either a regression model or a machine learning model to predict whether an employee will leave the company. The exemplar following this activity shows both approaches, but you only need to do one.

In your deliverables, you will include the model evaluation (and interpretation if applicable), a data visualization(s) of your choice that is directly related to the question you ask, ethical considerations, and the resources you used to troubleshoot and find answers or solutions.

2 PACE stages

2.1 Pace: Plan

Consider the questions in your PACE Strategy Document to reflect on the Plan stage.

In this stage, consider the following:

2.1.1 Understand the business scenario and problem

The HR department at Salifort Motors wants to take some initiatives to improve employee satisfaction levels at the company. They collected data from employees, but now they don't know what to do with it. They refer to you as a data analytics professional and ask you to provide data-driven

suggestions based on your understanding of the data. They have the following question: what's likely to make the employee leave the company?

Your goals in this project are to analyze the data collected by the HR department and to build a model that predicts whether or not an employee will leave the company.

If you can predict employees likely to quit, it might be possible to identify factors that contribute to their leaving. Because it is time-consuming and expensive to find, interview, and hire new employees, increasing employee retention will be beneficial to the company.

2.1.2 Familiarize yourself with the HR dataset

The dataset that you'll be using in this lab contains 15,000 rows and 10 columns for the variables listed below.

Note: you don't need to download any data to complete this lab. For more information about the data, refer to its source on [Kaggle](#).

Variable	Description
satisfaction_level	Employee-reported job satisfaction level [0–1]
last_evaluation	Score of employee's last performance review [0–1]
number_project	Number of projects employee contributes to
average_monthly_hours	Average number of hours employee worked per month
time_spend_company	How long the employee has been with the company (years)
Work_accident	Whether or not the employee experienced an accident while at work
left	Whether or not the employee left the company
promotion_last_5years	Whether or not the employee was promoted in the last 5 years
Department	The employee's department
salary	The employee's salary (U.S. dollars)

Reflect on these questions as you complete the plan stage.

- Who are your stakeholders for this project?
- What are you trying to solve or accomplish?
- What are your initial observations when you explore the data?
- What resources do you find yourself using as you complete this stage? (Make sure to include the links.)
- Do you have any ethical considerations in this stage?

[Double-click to enter your responses here.]

2.2 Step 1. Imports

- Import packages
- Load dataset

2.2.1 Import packages

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

2.2.2 Load dataset

Pandas is used to read a dataset called `HR_capstone_dataset.csv`. As shown in this cell, the dataset has been automatically loaded in for you. You do not need to download the .csv file, or provide more code, in order to access the dataset and proceed with this lab. Please continue with this activity by completing the following instructions.

```
[2]: df0 = pd.read_csv("HR_capstone_dataset.csv")

# Display first few rows
df0.head()
```

```
[2]:
```

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	\
0	0.38	0.53	2	157	
1	0.80	0.86	5	262	
2	0.11	0.88	7	272	
3	0.72	0.87	5	223	
4	0.37	0.52	2	159	

	time_spend_company	Work_accident	left	promotion_last_5years	Department	\
0	3	0	1	0	sales	
1	6	0	1	0	sales	
2	4	0	1	0	sales	
3	5	0	1	0	sales	
4	3	0	1	0	sales	

	salary
0	low
1	medium
2	medium
3	low
4	low

2.3 Step 2. Data Exploration (Initial EDA and data cleaning)

- Understand your variables
- Clean your dataset (missing data, redundant data, outliers)

2.3.1 Gather basic information about the data

```
[3]: # Gather basic information about the data
df0.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14999 entries, 0 to 14998
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   satisfaction_level      14999 non-null  float64
1   last_evaluation         14999 non-null  float64
2   number_project          14999 non-null  int64
3   average_monthly_hours  14999 non-null  int64
4   time_spend_company     14999 non-null  int64
5   Work_accident          14999 non-null  int64
6   left                   14999 non-null  int64
7   promotion_last_5years  14999 non-null  int64
8   Department              14999 non-null  object
9   salary                 14999 non-null  object
dtypes: float64(2), int64(6), object(2)
memory usage: 1.1+ MB
```

2.3.2 Gather descriptive statistics about the data

```
[4]: # Gather descriptive statistics about the data
df0.describe()
```

```
[4]:
```

	satisfaction_level	last_evaluation	number_project	\
count	14999.000000	14999.000000	14999.000000	
mean	0.612834	0.716102	3.803054	
std	0.248631	0.171169	1.232592	
min	0.090000	0.360000	2.000000	
25%	0.440000	0.560000	3.000000	
50%	0.640000	0.720000	4.000000	
75%	0.820000	0.870000	5.000000	
max	1.000000	1.000000	7.000000	

	average_monthly_hours	time_spend_company	Work_accident	left	\
count	14999.000000	14999.000000	14999.000000	14999.000000	
mean	201.050337	3.498233	0.144610	0.238083	

std	49.943099	1.460136	0.351719	0.425924
min	96.000000	2.000000	0.000000	0.000000
25%	156.000000	3.000000	0.000000	0.000000
50%	200.000000	3.000000	0.000000	0.000000
75%	245.000000	4.000000	0.000000	0.000000
max	310.000000	10.000000	1.000000	1.000000

	promotion_last_5years
count	14999.000000
mean	0.021268
std	0.144281
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	1.000000

2.3.3 Rename columns

As a data cleaning step, rename the columns as needed. Standardize the column names so that they are all in `snake_case`, correct any column names that are misspelled, and make column names more concise as needed.

```
[5]: # Display all column names
print(df0.columns)
```

```
Index(['satisfaction_level', 'last_evaluation', 'number_project',
       'average_monthly_hours', 'time_spend_company', 'Work_accident', 'left',
       'promotion_last_5years', 'Department', 'salary'],
      dtype='object')
```

```
[6]: # Rename columns as needed
df0 = df0.rename(columns={
    'Work_accident': 'work_accident',
    'average_monthly_hours': 'average_monthly_hours',
    'time_spend_company': 'tenure',
    'Department': 'department'
})

# Display all column names after the update
print(df0.columns)
```

```
Index(['satisfaction_level', 'last_evaluation', 'number_project',
       'average_monthly_hours', 'tenure', 'work_accident', 'left',
       'promotion_last_5years', 'department', 'salary'],
      dtype='object')
```

2.3.4 Check missing values

Check for any missing values in the data.

```
[7]: # Check for missing values
df0.isnull().sum()
```

```
[7]: satisfaction_level    0
last_evaluation          0
number_project           0
average_monthly_hours    0
tenure                   0
work_accident            0
left                     0
promotion_last_5years    0
department               0
salary                   0
dtype: int64
```

2.3.5 Check duplicates

Check for any duplicate entries in the data.

```
[8]: # Check for duplicates
print(df0.duplicated().sum())
```

3008

```
[9]: # Inspect some rows containing duplicates
df0[df0.duplicated()].head()
```

```
[9]:
```

	satisfaction_level	last_evaluation	number_project	\
396	0.46	0.57	2	
866	0.41	0.46	2	
1317	0.37	0.51	2	
1368	0.41	0.52	2	
1461	0.42	0.53	2	

	average_monthly_hours	tenure	work_accident	left	\
396	139	3	0	1	
866	128	3	0	1	
1317	127	3	0	1	
1368	132	3	0	1	
1461	142	3	0	1	

	promotion_last_5years	department	salary
396	0	sales	low

866	0	accounting	low
1317	0	sales	medium
1368	0	RandD	low
1461	0	sales	low

```
[10]: # Drop duplicates and save in new variable
df1 = df0.drop_duplicates(keep='first')

# Display first few rows of new dataframe
df1.head()
```

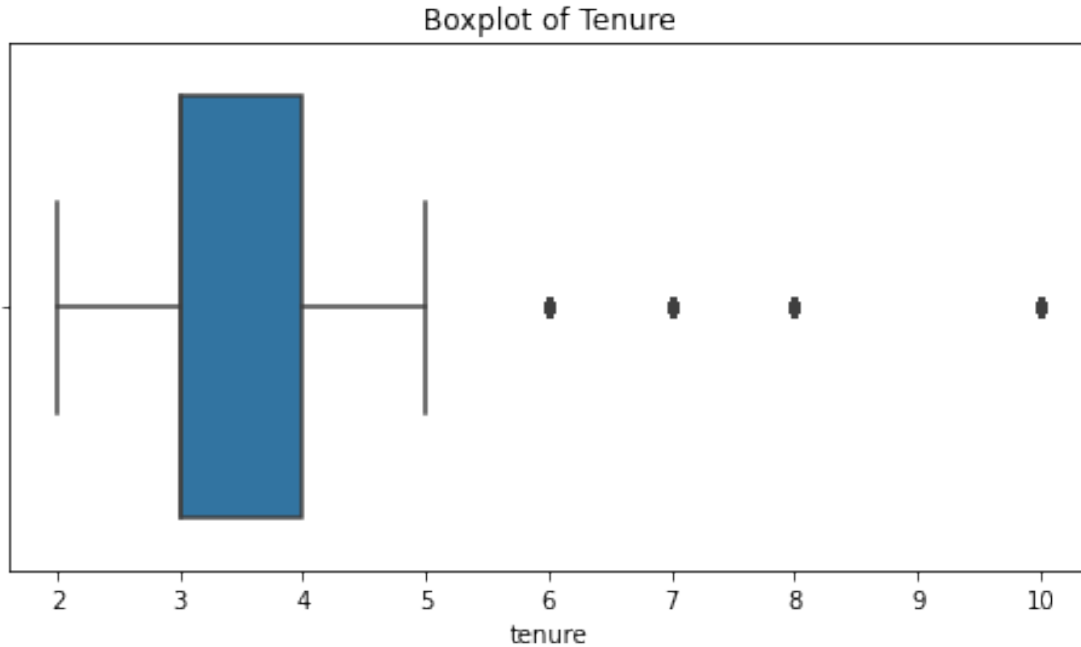
```
[10]: satisfaction_level  last_evaluation  number_project  average_monthly_hours  \
0                0.38                0.53                2                157
1                0.80                0.86                5                262
2                0.11                0.88                7                272
3                0.72                0.87                5                223
4                0.37                0.52                2                159
```

	tenure	work_accident	left	promotion_last_5years	department	salary
0	3	0	1	0	sales	low
1	6	0	1	0	sales	medium
2	4	0	1	0	sales	medium
3	5	0	1	0	sales	low
4	3	0	1	0	sales	low

2.3.6 Check outliers

Check for outliers in the data.

```
[11]: # Create a boxplot to visualize distribution of tenure and detect any outliers
plt.figure(figsize=(8,4))
sns.boxplot(x=df1['tenure'])
plt.title('Boxplot of Tenure')
plt.show()
```



```
[12]: # Determine the number of rows containing outliers
Q1 = df1['tenure'].quantile(0.25)
Q3 = df1['tenure'].quantile(0.75)
IQR = Q3 - Q1

outliers = df1[(df1['tenure'] < Q1 - 1.5*IQR) | (df1['tenure'] > Q3 + 1.5*IQR)]
print("Number of outlier rows:", len(outliers))
```

Number of outlier rows: 824

Certain types of models are more sensitive to outliers than others. When you get to the stage of building your model, consider whether to remove outliers, based on the type of model you decide to use.

3 pAce: Analyze Stage

- Perform EDA (analyze relationships between variables)

Reflect on these questions as you complete the analyze stage.

- What did you observe about the relationships between variables?
- What do you observe about the distributions in the data?
- What transformations did you make with your data? Why did you chose to make those decisions?
- What are some purposes of EDA before constructing a predictive model?

- What resources do you find yourself using as you complete this stage? (Make sure to include the links.)
- Do you have any ethical considerations in this stage?

[Double-click to enter your responses here.]

3.1 Step 2. Data Exploration (Continue EDA)

Begin by understanding how many employees left and what percentage of all employees this figure represents.

```
[13]: # Get numbers of people who left vs. stayed
print(df1['left'].value_counts())

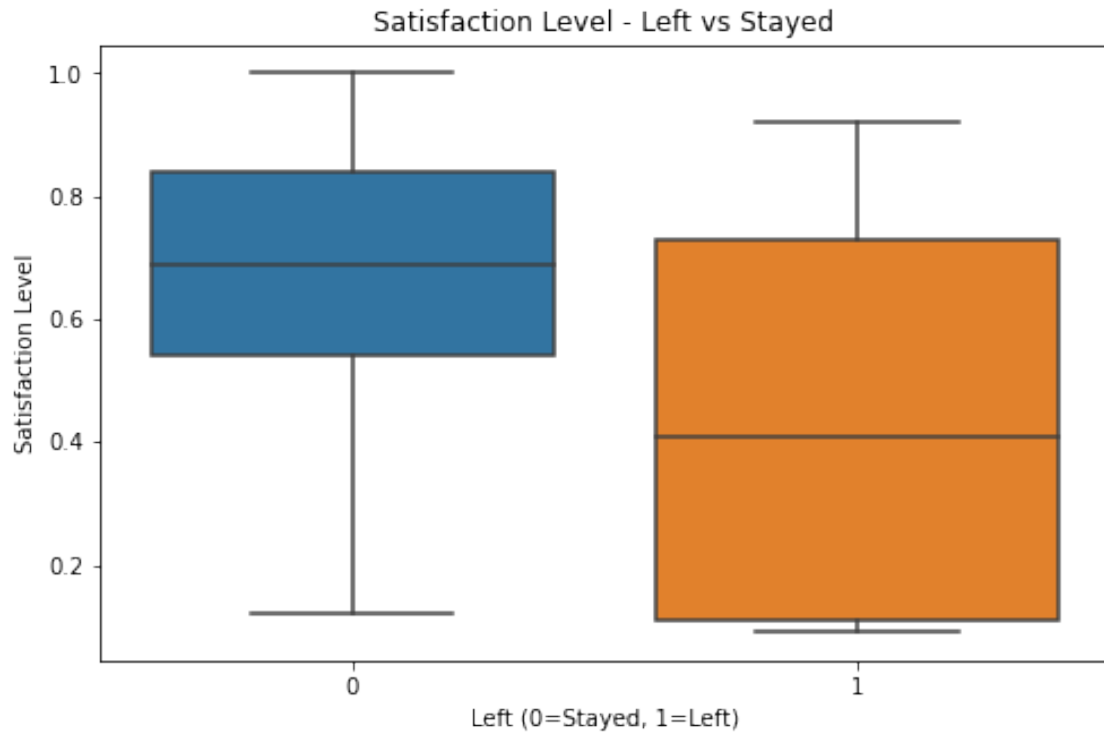
# Get percentages of people who left vs. stayed
print(df1['left'].value_counts(normalize=True)*100)
```

```
0    10000
1     1991
Name: left, dtype: int64
0     83.39588
1     16.60412
Name: left, dtype: float64
```

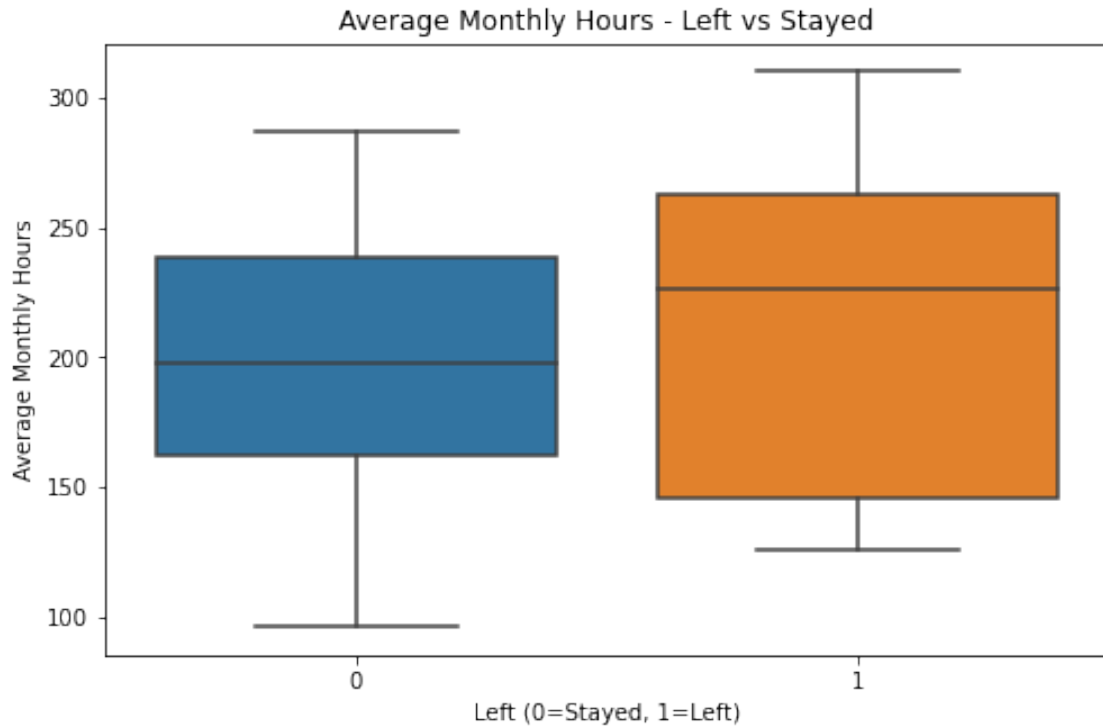
3.1.1 Data visualizations

Now, examine variables that you're interested in, and create plots to visualize relationships between variables in the data.

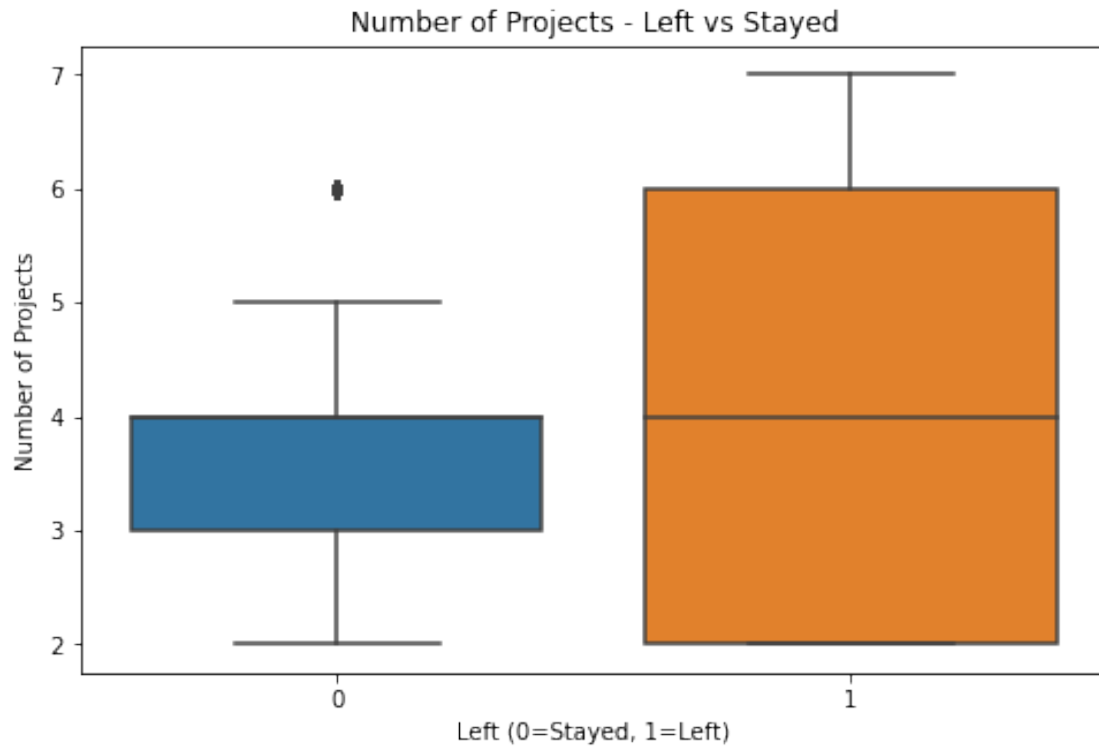
```
[14]: # Satisfaction level vs left
plt.figure(figsize=(8,5))
sns.boxplot(x='left', y='satisfaction_level', data=df1)
plt.title('Satisfaction Level - Left vs Stayed')
plt.xlabel('Left (0=Stayed, 1=Left)')
plt.ylabel('Satisfaction Level')
plt.show()
```



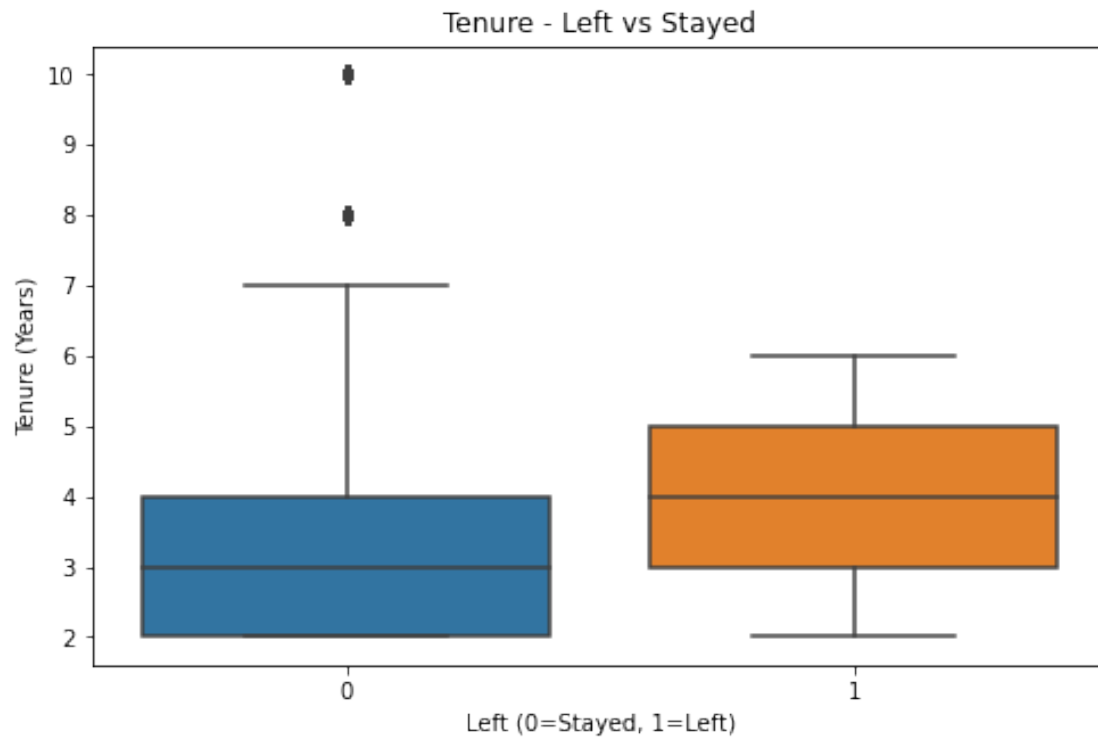
```
[15]: # Monthly hours vs left
plt.figure(figsize=(8,5))
sns.boxplot(x='left', y='average_monthly_hours', data=df1)
plt.title('Average Monthly Hours - Left vs Stayed')
plt.xlabel('Left (0=Stayed, 1=Left)')
plt.ylabel('Average Monthly Hours')
plt.show()
```



```
[16]: # Number of projects vs left
plt.figure(figsize=(8,5))
sns.boxplot(x='left', y='number_project', data=df1)
plt.title('Number of Projects - Left vs Stayed')
plt.xlabel('Left (0=Stayed, 1=Left)')
plt.ylabel('Number of Projects')
plt.show()
```



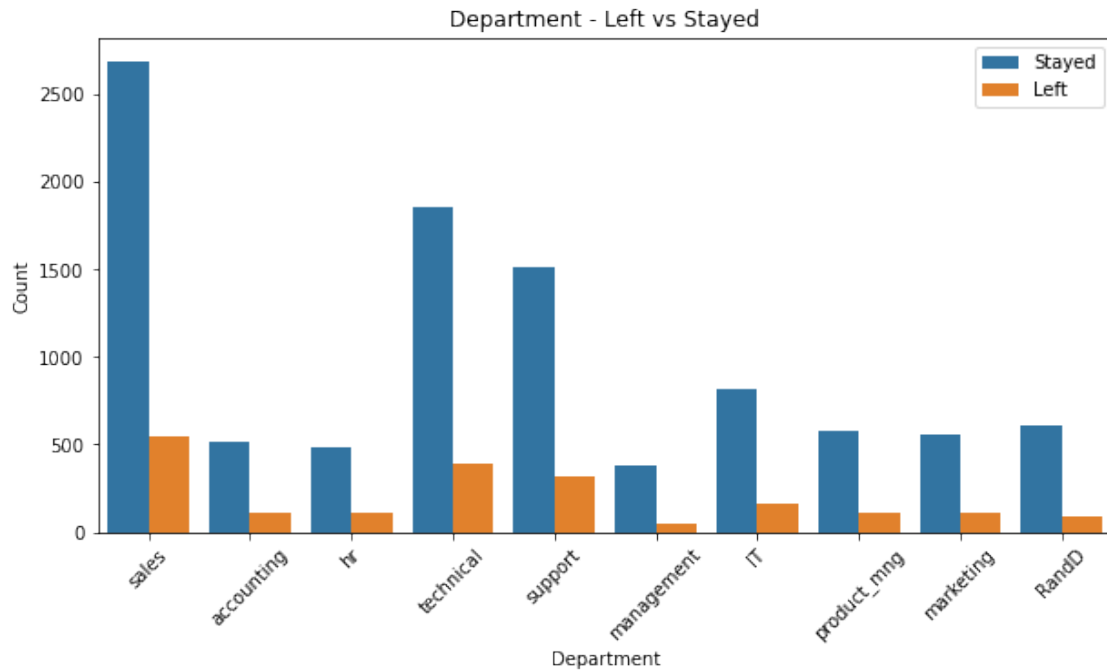
```
[17]: # Tenure vs left
plt.figure(figsize=(8,5))
sns.boxplot(x='left', y='tenure', data=df1)
plt.title('Tenure - Left vs Stayed')
plt.xlabel('Left (0=Stayed, 1=Left)')
plt.ylabel('Tenure (Years)')
plt.show()
```



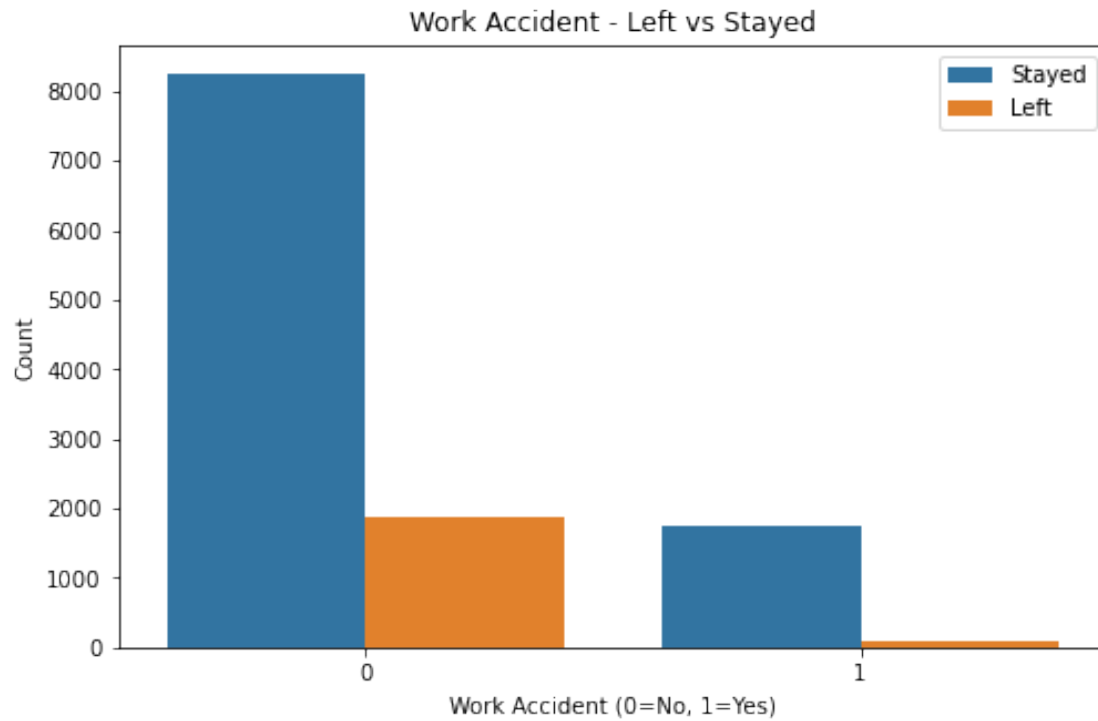
```
[18]: # Salary vs left
plt.figure(figsize=(8,5))
sns.countplot(x='salary', hue='left', data=df1)
plt.title('Salary Level - Left vs Stayed')
plt.xlabel('Salary Level')
plt.ylabel('Count')
plt.legend(['Stayed', 'Left'])
plt.show()
```



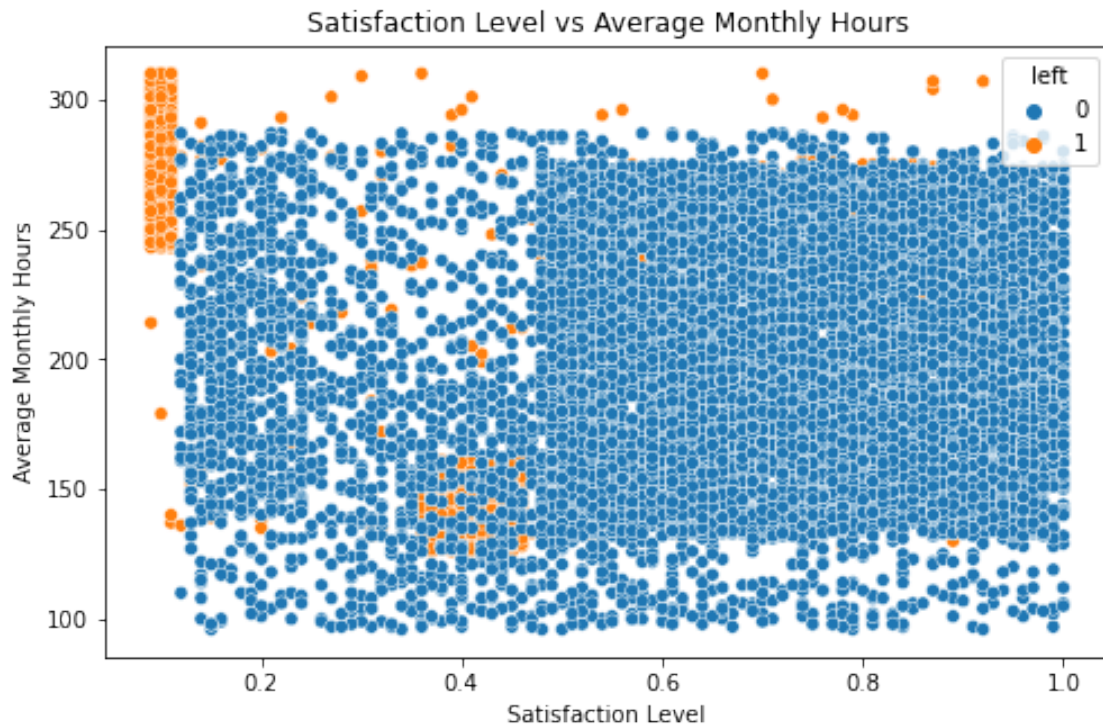
```
[19]: # Department vs left
plt.figure(figsize=(10,5))
sns.countplot(x='department', hue='left', data=df1)
plt.title('Department - Left vs Stayed')
plt.xlabel('Department')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.legend(['Stayed', 'Left'])
plt.show()
```



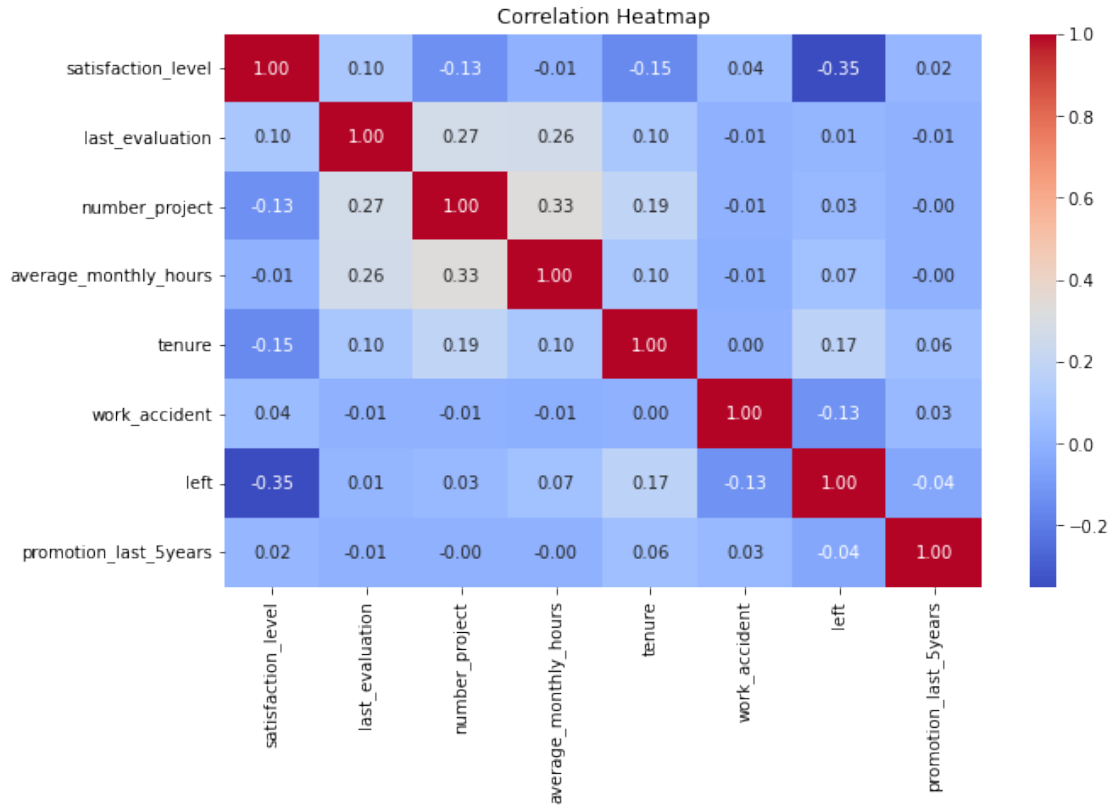
```
[20]: # Work accident vs left
plt.figure(figsize=(8,5))
sns.countplot(x='work_accident', hue='left', data=df1)
plt.title('Work Accident - Left vs Stayed')
plt.xlabel('Work Accident (0=No, 1=Yes)')
plt.ylabel('Count')
plt.legend(['Stayed', 'Left'])
plt.show()
```



```
[21]: # Scatter plot - satisfaction level vs average monthly hours
plt.figure(figsize=(8,5))
sns.scatterplot(x='satisfaction_level', y='average_monthly_hours',
                hue='left', data=df1)
plt.title('Satisfaction Level vs Average Monthly Hours')
plt.xlabel('Satisfaction Level')
plt.ylabel('Average Monthly Hours')
plt.show()
```



```
[31]: # Heatmap - correlation between variables
plt.figure(figsize=(10,6))
sns.heatmap(df1.corr(), annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



3.1.2 Insights

Employees with low satisfaction levels are more likely to leave. Overworked employees with high monthly hours and many projects also show higher turnover. Low salary and no promotion are strong indicators of employees leaving. Sales and technical departments have the highest turnover rates.

4 paCe: Construct Stage

- Determine which models are most appropriate
- Construct the model
- Confirm model assumptions
- Evaluate model results to determine how well your model fits the data

Recall model assumptions

Logistic Regression model assumptions - Outcome variable is categorical - Observations are independent of each other - No severe multicollinearity among X variables - No extreme outliers - Linear relationship between each X variable and the logit of the outcome variable - Sufficiently large sample size

Reflect on these questions as you complete the constructing stage.

- Do you notice anything odd?
- Which independent variables did you choose for the model and why?
- Are each of the assumptions met?
- How well does your model fit the data?
- Can you improve it? Is there anything you would change about the model?
- What resources do you find yourself using as you complete this stage? (Make sure to include the links.)
- Do you have any ethical considerations in this stage?

[Double-click to enter your responses here.]

4.1 Step 3. Model Building, Step 4. Results and Evaluation

- Fit a model that predicts the outcome variable using two or more independent variables
- Check model assumptions
- Evaluate the model

4.1.1 Identify the type of prediction task.

[Double-click to enter your responses here.]

4.1.2 Identify the types of models most appropriate for this task.

[Double-click to enter your responses here.]

4.1.3 Modeling

Add as many cells as you need to conduct the modeling process.

```
[22]: # Identify the type of prediction task
# This is a binary classification task - predicting if employee will leave (1)
      ↳ or stay (0)
```

```
# Import models
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
```

```
[23]: # Convert categorical columns to numbers
df2 = pd.get_dummies(df1, columns=['department', 'salary'])

# Define X (features) and y (target)
X = df2.drop('left', axis=1)
y = df2['left']
```

```
# Split data into train and test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳random_state=42)

print("Training size:", X_train.shape)
print("Testing size:", X_test.shape)
```

Training size: (9592, 20)
Testing size: (2399, 20)

```
[24]: # Build logistic regression model
model = LogisticRegression(max_iter=500)

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

print("Model training done!")
```

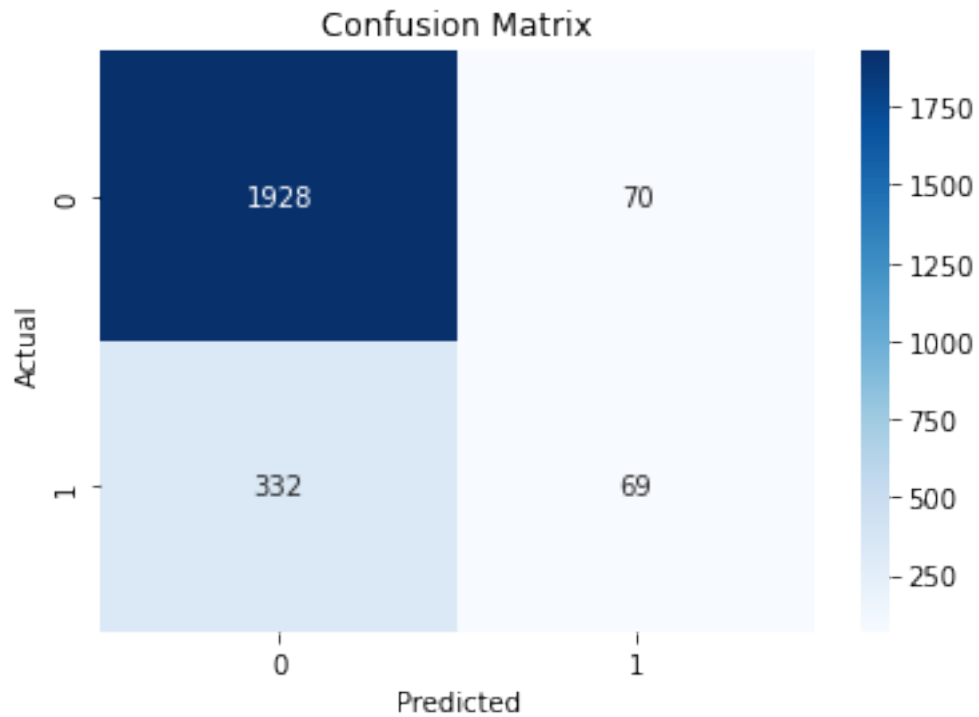
Model training done!

```
[25]: # Evaluate the model
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

Classification Report:

	precision	recall	f1-score	support
0	0.85	0.96	0.91	1998
1	0.50	0.17	0.26	401
accuracy			0.83	2399
macro avg	0.67	0.57	0.58	2399
weighted avg	0.79	0.83	0.80	2399

```
[26]: # Confusion matrix
sns.heatmap(confusion_matrix(y_test, y_pred),
            annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



```
[27]: # Try Random Forest for better results
from sklearn.ensemble import RandomForestClassifier

# Build random forest model
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)

# Train the model
rf_model.fit(X_train, y_train)

# Make predictions
rf_pred = rf_model.predict(X_test)

print("Random Forest training done!")
```

Random Forest training done!

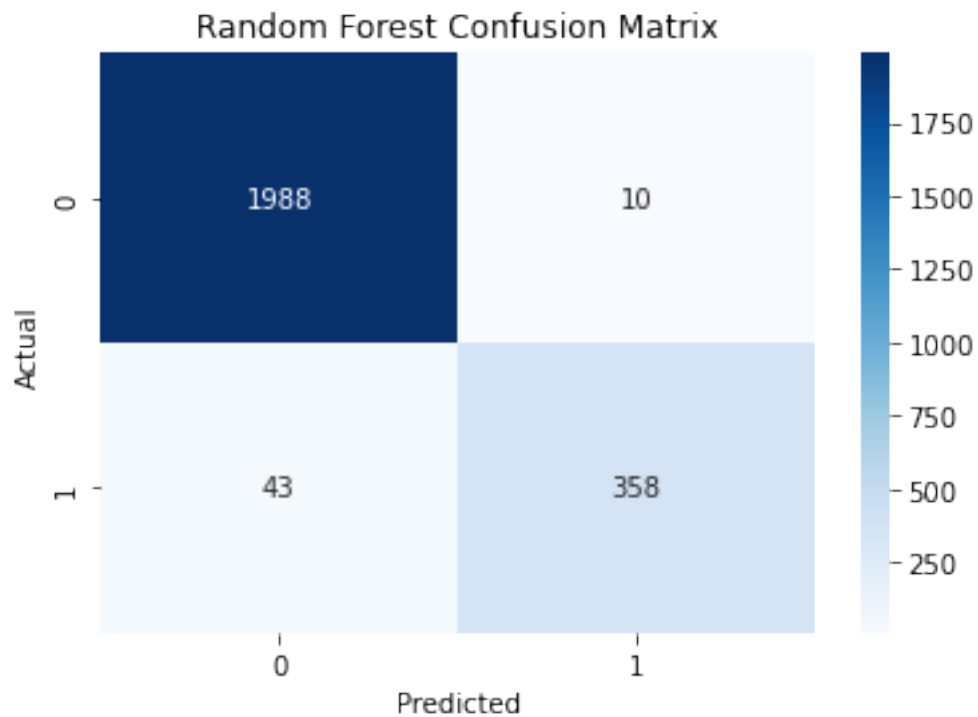
```
[28]: # Evaluate random forest model
print("Classification Report:")
print(classification_report(y_test, rf_pred))
```

Classification Report:

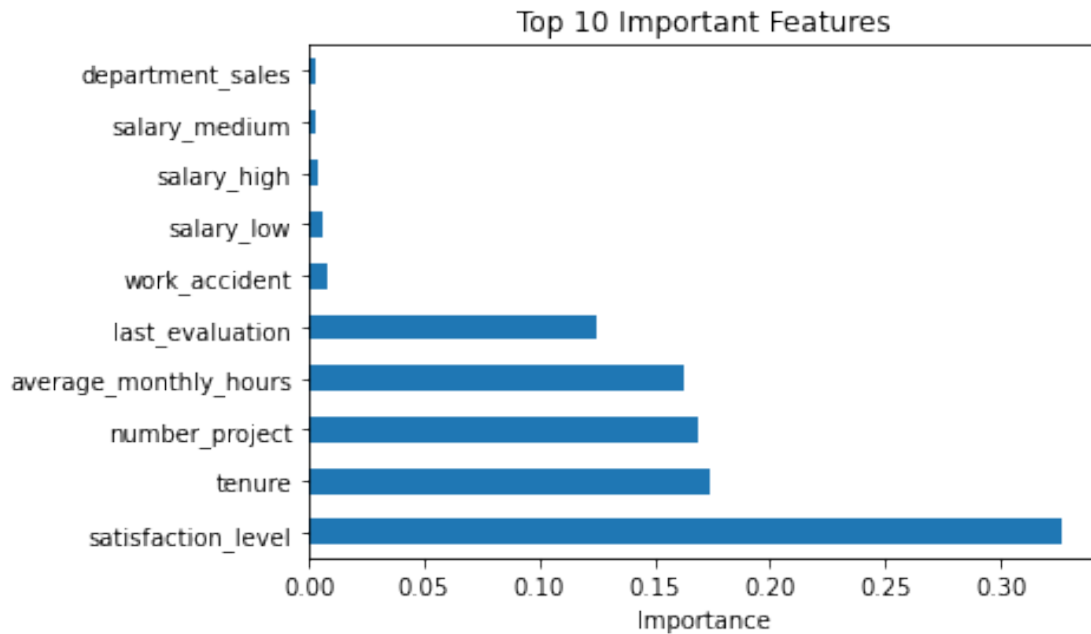
	precision	recall	f1-score	support
0	0.98	0.99	0.99	1998

	1	0.97	0.89	0.93	401
accuracy				0.98	2399
macro avg		0.98	0.94	0.96	2399
weighted avg		0.98	0.98	0.98	2399

```
[29]: # Confusion matrix for random forest
sns.heatmap(confusion_matrix(y_test, rf_pred),
             annot=True, fmt='d', cmap='Blues')
plt.title('Random Forest Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



```
[30]: # Feature importance
feature_importance = pd.Series(rf_model.feature_importances_,
                               index=X.columns)
feature_importance.nlargest(10).plot(kind='barh')
plt.title('Top 10 Important Features')
plt.xlabel('Importance')
plt.show()
```



5 pacE: Execute Stage

- Interpret model performance and results
- Share actionable steps with stakeholders

Recall evaluation metrics

- **AUC** is the area under the ROC curve; it's also considered the probability that the model ranks a random positive example more highly than a random negative example.
- **Precision** measures the proportion of data points predicted as True that are actually True, in other words, the proportion of positive predictions that are true positives.
- **Recall** measures the proportion of data points that are predicted as True, out of all the data points that are actually True. In other words, it measures the proportion of positives that are correctly classified.
- **Accuracy** measures the proportion of data points that are correctly classified.
- **F1-score** is an aggregation of precision and recall.

Reflect on these questions as you complete the executing stage.

- What key insights emerged from your model(s)?
- What business recommendations do you propose based on the models built?
- What potential recommendations would you make to your manager/company?
- Do you think your model could be improved? Why or why not? How?
- Given what you know about the data and the models you were using, what other questions could you address for the team?

- What resources do you find yourself using as you complete this stage? (Make sure to include the links.)
- Do you have any ethical considerations in this stage?

Double-click to enter your responses here.

5.1 Step 4. Results and Evaluation

- Interpret model
- Evaluate model performance using metrics
- Prepare results, visualizations, and actionable steps to share with stakeholders

5.1.1 Summary of model results

Two models were built to predict employee turnover. Logistic Regression gave 83% accuracy but low recall of 17%. Random Forest performed much better with 98% accuracy and 89% recall. Random Forest was selected as the final model. The most important factors affecting employee turnover are satisfaction level, tenure, number of projects, and average monthly hours.

5.1.2 Conclusion, Recommendations, Next Steps

Employees with low satisfaction, high workload, and no promotion are most likely to leave. The company should focus on improving employee satisfaction, reducing overwork, and offering more promotions and salary growth. The Random Forest model can be used to identify at-risk employees early so HR can take action before they leave.

Congratulations! You’ve completed this lab. However, you may not notice a green check mark next to this item on Coursera’s platform. Please continue your progress regardless of the check mark. Just click on the “save” icon at the top of this notebook to ensure your work has been logged.